

SOLIDWORKS Document Manager API (.NET) — Examples (4/4): Get_PNG_Preview_Bitmap_and_Stream_for_Co- Write_Parasolid_Partition_Stream_to_File

SOLIDWORKS API Documentation (converted from CHM)

Get PNG Preview Bitmap and Stream for Configuration Example (C#)

This example shows how to get the name of the stream and the PNG preview bitmap for a configuration in a part.

NOTE: To get the name of the stream and a preview bitmap of the sheet active when the drawing was last saved, use ISwDMDocument10::GetPreviewBitmap and ISwDMDocument10::PreviewStreamName. To get the name of the streams and the preview bitmaps for all sheets in a drawing, use ISwDMSheet2::GetPreviewPNGBitmap and ISwDMSheet2::PreviewPNGStreamName.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
// and verify that the required DLLs are registered.
// 2. Create a C# macro in SOLIDWORKS.
// 3. Copy Main class into SolidWorksMacro.cs.
// a. Specify your_license_key.
// b. Verify that the specified file to open exists.
// c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
// 1. Right-click the project in the Project Explorer.
// 2. Click Add Reference.
// 3. Click Browse.
// 4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
// d. Add references to System.Drawing and stdole.
// 4. Create a class and copy Class1 into Class1.cs.
// 5. Verify that C:\temp exists.
// 6. Open the Immediate Window.
//
// Postconditions:
// 1. Creates a preview bitmap of the Default configuration.
// 2. Right-click C:\temp\Default.PNG and click Preview.
// 3. Examine the Immediate Window.
//-----
//Main class
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System.Drawing;
using stdole;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";
            SwDMClassFactory swClassFact = default(SwDMClassFactory);
            SwDMAApplication swDocMgr = default(SwDMAApplication);
            SwDMDocument swDoc = default(SwDMDocument);
            SwDMConfigurationMgr swCfgMgr = default(SwDMConfigurationMgr);
            string[] vCfgNameArr = null;
            SwDMConfiguration7 swCfg = default(SwDMConfiguration7);
            SwDmDocumentType nDocType = 0;
            SwDmDocumentOpenError nRetVal = 0;
            SwDmPreviewError nError = 0;

            // Determine type of SOLIDWORKS file based on file extension
            if (sDocFileName.EndsWith(".sldprt"))
            {
                nDocType = SwDmDocumentType.swDmDocumentPart;
            }
            else if (sDocFileName.EndsWith(".sldasm"))
            {
                nDocType = SwDmDocumentType.swDmDocumentAssembly;
            }
            else if (sDocFileName.EndsWith(".slddrw"))
            {
                nDocType = SwDmDocumentType.swDmDocumentDrawing;
            }
            else
            {
                // Not a SOLIDWORKS file, so cannot open
                nDocType = SwDmDocumentType.swDmDocumentUnknown;
                return;
            }

            // Because drawing documents do not have configurations,
            // only continue running the macro if the document
            // is a part or assembly document
            if ((nDocType != SwDmDocumentType.swDmDocumentDrawing))
            {
                swClassFact = new SwDMClassFactory();
                swDocMgr = (SwDMAApplication)swClassFact.GetApplication("your_license_key");
                swDoc = (SwDMDocument)swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal);

                swCfgMgr = swDoc.ConfigurationManager;

                Debug.Print("File = " + swDoc.FullName);
                Debug.Print("Active configuration name = " + swCfgMgr.GetActiveConfigurationName());
                vCfgNameArr = (string[])swCfgMgr.GetConfigurationNames();

                foreach (string vCfgName in vCfgNameArr)
                {
                    swCfg = (SwDMConfiguration7)swCfgMgr.GetConfigurationByName(vCfgName);
                    // SwDMConfiguration7::GetPreviewPNGBitmap throws an unmanaged COM exception
                    // for out-of-process C# console applications
                    // Use the following code in SOLIDWORKS C# macros and add-ins
                    object objBitmap = swCfg.GetPreviewPNGBitmap(out nError);
                    System.Drawing.Image imgPreview = PictureDispConverter.Convert(objBitmap);
                    imgPreview.Save("C:\\temp\\" + vCfgName + ".PNG", System.Drawing.Imaging.ImageFormat.Png);
                    imgPreview.Dispose();

                    Debug.Print("    " + vCfgName);
                }
            }
        }
    }
}
```

```

        Debug.Print("    PNG preview stream = " + swCfg.PreviewPNGStreamName);
    }
    Debug.Print(" ");
    swDoc.CloseDoc();
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

}

//Class1
public class PictureDispConverter : System.Windows.Forms.AxHost
{
    public PictureDispConverter()
    {
        : base("56174C86-1546-4778-8EE6-B6AC606875E7")
    }

    public static System.Drawing.Image Convert(object objIDispImage)
    {
        System.Drawing.Image objPicture = default(System.Drawing.Image);
        objPicture = (System.Drawing.Image)System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage);
        return objPicture;
    }
}
}

```

Get PNG Preview Bitmap and Stream for Configuration Example (VB.NET)

This example shows how to get the name of the stream and the PNG preview bitmap for a configuration.

NOTE: To get the name of the stream and a preview bitmap of the sheet active when the drawing was last saved, use ISwDMDocument10::GetPreviewBitmap and ISwDMDocument10::PreviewStreamName. To get the name of the streams and the preview bitmaps for all sheets in a drawing, use ISwDMSheet2::GetPreviewPNGBitmap and ISwDMSheet2::PreviewPNGStreamName.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
'   and verify that the required DLLs are registered.
' 2. Create a VB.NET console application in Microsoft Visual Studio.
' 3. Copy Module1 into Module1.vb.
'   a. Specify your_license_key.
'   b. Verify that the specified file to open exists.
'   c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
'       1. Right-click the solution in the Solution Explorer.
'       2. Click Add Reference.
'       3. Click Browse.
'       4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'   d. Add references to System.Windows.Forms, System.Drawing, and stdole.
' 4. Create a class and copy Class1 into Class1.vb.
' 5. Verify that C:\temp exists.
' 6. Open the Immediate Window.
'-----
' Postconditions:
' 1. Creates a preview bitmap of the Default configuration.
' 2. Right-click C:\temp\Default.PNG and click Preview.
' 3. Examine the Immediate Window.
'-----
Module1
Imports SolidWorks.Interop.swdocumentmgr
Imports System.Drawing.Image
Imports stdole
Imports System
Imports System.Diagnostics

Module Module1

    Sub Main()
        Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMApplication
        Dim swDoc As SwDMDocument
        Dim swCfgMgr As SwDMConfigurationMgr
        Dim vCfgNameArr As Object
        Dim vCfgName As Object
        Dim swCfg As SwDMConfiguration7
        Dim nDocType As Integer
        Dim nRetVal As Integer

        ' Determine type of SOLIDWORKS file based on file extension
        If InStr(LCase(sDocFileName), "sldprt") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentPart
        ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentAssembly
        ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentDrawing
        Else
            ' Probably not a SOLIDWORKS file,
            ' so cannot open
            nDocType = SwDmDocumentType.swDmDocumentUnknown
            Exit Sub
        End If

        ' Because drawing documents do not have configurations,
        ' only continue running the macro if the document
        ' is a part or assembly document
        If (nDocType <> SwDmDocumentType.swDmDocumentDrawing) Then

            swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
            swDocMgr = swClassFact.GetApplication("your_license_key")
            swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)
            swCfgMgr = swDoc.ConfigurationManager

            Debug.Print("File = " & swDoc.FullName)
            Debug.Print("Active configuration name = " & swCfgMgr.GetActiveConfigurationName)
            vCfgNameArr = swCfgMgr.GetConfigurationNames

            For Each vCfgName In vCfgNameArr
                swCfg = swCfgMgr.GetConfigurationByName(vCfgName)
                Dim objBitmap As Object = swCfg.GetPreviewBitmap(nRetVal)
                Dim imgPreview As System.Drawing.Image = PictureDispConverter.Convert(objBitmap)
                imgPreview.Save("C:\temp\\" & vCfgName & ".PNG", Drawing.Imaging.ImageFormat.Bmp)
                imgPreview.Dispose()

                Debug.Print(" " & vCfgName)
                Debug.Print(" PNG preview stream = " & swCfg.PreviewPNGStreamName)
                Debug.Print(" ")
            Next
            swDoc.CloseDoc()
        End If

    End Sub

End Module

'Class1
Public Class PictureDispConverter

    Inherits System.Windows.Forms.AxHost

```

```
Public Sub New()  
    MyBase.New("56174C86-1546-4778-8EE6-B6AC606875E7")  
End Sub  
  
Public Shared Function Convert(ByVal objDispImage As Object) As System.Drawing.Image  
    Dim objPicture As System.Drawing.Image  
    objPicture = CType(System.Windows.Forms.AxHost.GetPictureFromIPicture(objDispImage), System.Drawing.Image)  
    Return objPicture  
End Function  
  
End Class
```

Get Preview Bitmap of Drawing Sheet Example (C#)

This example shows how to get a BMP preview bitmap of the sheet that was active when the drawing was last saved.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
// and verify that the required DLLs are registered.
// 2. Create a C# macro in SOLIDWORKS.
// 3. Copy Main into SolidWorksMacro.cs.
// a. Specify your_license_key.
// b. Verify that the specified file to open exists.
// c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
// 1. Right-click the project in the Project Explorer.
// 2. Click Add Reference.
// 3. Click Browse.
// 4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
// d. Add references to System.Drawing and stdole.
// 4. Create a class and copy Class1 into Class1.cs.
// 5. Verify that c:\temp exists.
// 6. Open the Immediate Window.
//
// Postconditions:
// 1. Creates a preview bitmap of the sheet active when the drawing was last saved.
// 2. Right-click c:\temp\foodprocessor.bmp and click Preview.
// 3. Examine the Immediate Window.
//-----
//Main
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using SolidWorks.Interop.swdocumentmgr;
using System.Drawing;
using stdole;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            const string sLicenseKey = "your_license_key"
            const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            const string sExtractDir = "c:\\temp\\";
            const string sFilename = "foodprocessor";
            SwDMClassFactory swClassFact = default(SwDMClassFactory);
            SwDMAApplication swDocMgr = default(SwDMAApplication);
            SwDMDocument swDoc = default(SwDMDocument);
            SwDMDocument10 swDoc10 = default(SwDMDocument10);
            SwDmDocumentType nDocType = 0;
            SwDmDocumentOpenError nRetVal = 0;
            SwDmPreviewError nError = 0;

            // Determine type of SOLIDWORKS file based on file extension
            if (sDocFileName.EndsWith(".sldprt"))
            {
                nDocType = SwDmDocumentType.swDmDocumentPart;
            }
            else if (sDocFileName.EndsWith(".sldasm"))
            {
                nDocType = SwDmDocumentType.swDmDocumentAssembly;
            }
            else if (sDocFileName.EndsWith(".slddrw"))
            {
                nDocType = SwDmDocumentType.swDmDocumentDrawing;
            }
            else
            {
                // Probably not a SOLIDWORKS file,
                // so cannot open
                nDocType = SwDmDocumentType.swDmDocumentUnknown;
                return;
            }

            swClassFact = new SwDMClassFactory();
            swDocMgr = (SwDMAApplication)swClassFact.GetApplication(sLicenseKey);
            swDoc = (SwDMDocument)swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal);
            Debug.Print("File = " + swDoc.FullName);
            Debug.Print(" Version      = " + swDoc.GetVersion());
            Debug.Print(" Author        = " + swDoc.Author);
            Debug.Print(" Comments     = " + swDoc.Comments);
            Debug.Print(" CreationDate  = " + swDoc.CreationDate);
            Debug.Print(" Keywords     = " + swDoc.Keywords);
            Debug.Print(" LastSavedBy  = " + swDoc.LastSavedBy);
            Debug.Print(" LastSavedDate = " + swDoc.LastSavedDate);
            Debug.Print(" Subject      = " + swDoc.Subject);
            Debug.Print(" Title       = " + swDoc.Title);

            swDoc10 = (SwDMDocument10)swDoc;
            // SwDMDocument10::GetPreviewBitmap throws an unmanaged COM exception
            // for out-of-process C# console applications
            // Use the following code in SOLIDWORKS C# macros and add-ins
            object objBitMap = swDoc10.GetPreviewBitmap(out nError);
            System.Drawing.Image imgPreview = PictureDispConverter.Convert(objBitMap);
            imgPreview.Save(sExtractDir + sFilename + ".bmp", System.Drawing.Imaging.ImageFormat.Bmp);
            imgPreview.Dispose();
        }
    }
}
```

```

        Debug.Print("    Preview stream    = " + swDoc10.PreviewStreamName);
    }
    /// <summary>
    ///     The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

//Class1
public class PictureDispConverter : System.Windows.Forms.AxHost
{
    public PictureDispConverter()
        : base("56174C86-1546-4778-8EE6-B6AC606875E7")
    {
    }

    public static System.Drawing.Image Convert(object objIDispImage)
    {
        System.Drawing.Image objPicture = default(System.Drawing.Image);
        objPicture = (System.Drawing.Image)System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage);
        return objPicture;
    }
}
}

```

Get Preview Bitmap of Drawing Sheet Example (VB.NET)

This example shows how to get a BMP preview bitmap of the sheet that was active when the drawing was last saved.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
'    and verify that the required DLLs are registered.
' 2. Create a VB.NET application in Microsoft Visual Studio.
' 3. Copy Module1 into Module1.vb.
'    a. Specify your_license_key.
'    b. Verify that the specified file to open exists.
'    c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
'        1. Right-click the solution in the Solution Explorer.
'        2. Click Add Reference.
'        3. Click Browse.
'        4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'    d. Add references to System.Windows.Forms, System.Drawing, and stdole.
' 4. Create a class and copy Class1 into Class1.vb.
' 5. Verify that c:\temp exists.
' 6. Open the Immediate Window.
'-----
' Postconditions:
' 1. Creates a preview bitmap of the sheet active when the drawing was last saved.
' 2. Right-click c:\temp\foodprocessor.bmp and click Preview.
' 3. Examine the Immediate Window.
'-----
'Module1

Imports SolidWorks.Interop.swdocumentmgr
Imports System.Drawing
Imports stdole
Imports System
Imports System.Diagnostics
Imports System.Windows.Forms

Module Module1

    Sub Main()

        Const sLicenseKey As String = "your_license_key"
        Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        Const sExtractDir As String = "c:\temp\"
        Const sFilename As String = "foodprocessor"
        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMApplication
        Dim swDoc As SwDMDocument
        Dim swDoc10 As SwDMDocument10
        Dim nDocType As Integer
        Dim nRetVal As Integer

        ' Determine type of SOLIDWORKS file based on file extension
        If InStr(LCase(sDocFileName), "sldprt") > 0 Then
            nDocType = SwDMDocumentType.swDMDocumentPart
        ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
            nDocType = SwDMDocumentType.swDMDocumentAssembly
        ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
            nDocType = SwDMDocumentType.swDMDocumentDrawing
        Else
            ' Probably not a SOLIDWORKS file,
            ' so cannot open
            nDocType = SwDMDocumentType.swDMDocumentUnknown
            Exit Sub
        End If

        swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
        swDocMgr = swClassFact.GetApplication(sLicenseKey)
        swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)
        Debug.Print("File = " & swDoc.FullName)
        Debug.Print("Version      = " & swDoc.GetVersion)
        Debug.Print("Author        = " & swDoc.Author)
        Debug.Print("Comments     = " & swDoc.Comments)
        Debug.Print("CreationDate  = " & swDoc.CreationDate)
        Debug.Print("Keywords     = " & swDoc.Keywords)
        Debug.Print("LastSavedBy  = " & swDoc.LastSavedBy)
        Debug.Print("LastSavedDate = " & swDoc.LastSavedDate)
        Debug.Print("Subject      = " & swDoc.Subject)
        Debug.Print("Title       = " & swDoc.Title)

        swDoc10 = swDoc
        Dim objBitmap As Object = swDoc10.GetPreviewBitmap(nRetVal)
        Dim imgPreview As System.Drawing.Image = PictureDispConverter.Convert(objBitmap)
        imgPreview.Save(sExtractDir + sFilename + ".bmp", Drawing.Imaging.ImageFormat.Bmp)
        imgPreview.Dispose()

        Debug.Print("    Preview stream = " & swDoc10.PreviewStreamName)
    End Sub

End Module

'Class1
Public Class PictureDispConverter

    Inherits System.Windows.Forms.AxHost

    Public Sub New()
        MyBase.New("56174C86-1546-4778-8EE6-B6AC606875E7")
    End Sub

    Public Shared Function Convert(ByVal objIDispImage As Object) As System.Drawing.Image
        Dim objPicture As System.Drawing.Image
        objPicture = CType(System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage), System.Drawing.Image)
        Return objPicture
    End Function

End Class

```

Get Preview Bitmaps of Drawing Sheets Example (C#)

This example shows how to get PNG preview bitmaps of the sheets in a drawing document.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
//    and verify that the required DLLs are registered.
// 2. Create a C# macro in SOLIDWORKS.
// 3. Copy Main class into SolidWorksMacro.cs.
//    a. Specify your_license_key.
//    b. Verify that the specified file to open exists.
//    c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
//        1. Right-click the project in the Project Explorer.
//        2. Click Add Reference.
//        3. Click Browse.
//        4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
//    d. Add references to System.Drawing and stdole.
// 4. Create a class and copy Class1 into Class1.cs.
// 5. Verify that c:\temp exists.
// 6. Open the Immediate Window.
//
// Postconditions:
// 1. Creates preview bitmaps of the sheets in the drawing document.
// 2. Right-click each c:\temp\Sheetn.png and click Preview.
// 3. Examine the Immediate Window.
//-----
//Main class
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System.Drawing;
using stdole;
using System.Runtime.InteropServices;
using System;
using System.Windows.Forms;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            SwDMClassFactory swClassFact = default(SwDMClassFactory);
            SwDMAApplication swDocMgr = default(SwDMAApplication);
            SwDMDocument10 swDoc = default(SwDMDocument10);
            string sDocFileName = null;
            SwDmDocumentType nDocType = default(SwDmDocumentType);
            SwDmDocumentOpenError nRetVal = default(SwDmDocumentOpenError);
            string sLicenseKey = null;

            // Specify your license key
            sLicenseKey = "your_license_key"

            // If the following drawing document does not exist,
            // then substitute the name of a drawing document that does
            sDocFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw";
            nDocType = SwDmDocumentType.swDmDocumentDrawing;

            swClassFact = new SwDMClassFactory();
            swDocMgr = (SwDMAApplication)swClassFact.GetApplication(sLicenseKey);
            swDoc = (SwDMDocument10)swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal);
            if ((swDoc == null))
            {
                MessageBox.Show("Unable to open document.");
            }

            object[] Sheets = null;
            SwDMSheet2 Sheet = default(SwDMSheet2);
            SwDmPreviewError nError;
            byte[] PreviewPNGByteArray = null;

            // Get the sheets in the drawing document
            Sheets = (object[])swDoc.GetSheets();
            int nbrSheets = 0;
            nbrSheets = Sheets.Length;
            string snbrSheets = Convert.ToString(nbrSheets);
            Debug.Print("Number of sheets: " + snbrSheets);
            Debug.Print(" ");

            int i = 0;
            for (i = 0; i < Sheets.Length; i++)
            {
                Sheet = (SwDMSheet2)Sheets[i];
                Debug.Print("Name of sheet: " + (Sheet.Name));
                Debug.Print("Name of preview PNG's stream: " + (Sheet.PreviewPNGStreamName));
                // SwDMSheet2::GetPreviewPNGBitmap throws an unmanaged COM exception
                // for out-of-process C# console applications
                // Use the following code in SOLIDWORKS C# macros and add-ins
                object objBitMap = Sheet.GetPreviewPNGBitmap(out nError);
                if (nError == 0)
                {
                    // For each sheet, convert the picture to an
                    // image and save it as .png file
                    System.Drawing.Image imgPreview = PictureDispConverter.Convert(objBitMap);
                    imgPreview.Save("c:\\temp\\" + Sheet.Name + ".png", System.Drawing.Imaging.ImageFormat.Png);
                    imgPreview.Dispose();

                    // Get the preview's PNG byte array
                    PreviewPNGByteArray = (byte[])Sheet.GetPreviewPNGBitmapBytes(out nError);
                    int nbrPreviewPNGBitmapBytes = 0;
                    nbrPreviewPNGBitmapBytes = PreviewPNGByteArray.Length;
                    string snbrPreviewPNGBitmapBytes = Convert.ToString(nbrPreviewPNGBitmapBytes);
                    Debug.Print("Number of bytes in preview's PNG byte array: " + snbrPreviewPNGBitmapBytes);
                }
            }
        }
    }
}
```

```

        Debug.Print("");
    }
    else
    {
        switch (nError)
        {
            case SwDmPreviewError.SwDmPreviewErrorNoPreview:
                Debug.Print("Error: No preview data stored with document.");
                break;
            case SwDmPreviewError.SwDmPreviewErrorMakeBmpFail:
                Debug.Print("Error: Failed to make the bitmap.");
                break;
        }
    }
}

swDoc.CloseDoc();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

//Class1

public class PictureDispConverter : System.Windows.Forms.AxHost
{
    public PictureDispConverter()
        : base("56174C86-1546-4778-8EE6-B6AC606875E7")
    {
    }

    public static System.Drawing.Image Convert(object objIDispImage)
    {
        System.Drawing.Image objPicture = default(System.Drawing.Image);
        objPicture = (System.Drawing.Image)System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage);
        return objPicture;
    }
}
}

```

Get Preview Bitmaps of Drawing Sheets Example (VB.NET)

This example shows how to get PNG preview bitmaps of the sheets in a drawing document.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
'    and verify that the required DLLs are registered.
' 2. Create a VB.NET console application in Microsoft Visual Studio.
' 3. Copy Module1 into Module1.vb.
'    a. Specify your_license_key.
'    b. Verify that the specified file to open exists.
'    c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
'        1. Right-click the solution in the Solution Explorer.
'        2. Click Add Reference.
'        3. Click Browse.
'        4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'    d. Add references to System.Windows.Forms, System.Drawing, and stdole.
' 4. Create a class and copy Class1 into Class1.vb.
' 5. Verify that c:\temp exists.
' 6. Open the Immediate Window.
'
' Postconditions:
' 1. Creates preview bitmaps of the sheets in the drawing document.
' 2. Right-click each c:\temp\Sheetn.png and click Preview.
' 3. Examine the Immediate Window.
'-----
'Module1

Imports System
Imports System.Diagnostics
Imports SolidWorks.Interop.swdocumentmgr
Imports System.Drawing
Imports stdole
Imports System.Windows.Forms

Module Module1

    Sub Main()

        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMApplication
        Dim swDoc As SwDMDocument10
        Dim sDocFileName As String
        Dim nDocType As SwDMDocumentType
        Dim nRetVal As SwDMDocumentOpenError
        Dim sLicenseKey As String

        ' Specify your license key
        sLicenseKey = "your_license_key"

        ' If the following drawing document does not exist,
        ' then substitute the name of a drawing document that does
        sDocFileName = "C:\Users\Public\Documents\SOLIDWORKS\2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        nDocType = SwDMDocumentType.swDMDocumentDrawing

        swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
        swDocMgr = swClassFact.GetApplication(sLicenseKey)
        swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)
        If (swDoc Is Nothing) Then
            MsgBox("Unable to open document.")
        End If

        Dim Sheets As Object
        Dim Sheet As SwDMSheet2
        Dim nError As Integer
        Dim PreviewPNGByteArray As Object

        ' Get the sheets in the drawing document
        Sheets = swDoc.GetSheets
        Dim nbrSheets As Integer
        nbrSheets = CLng(UBound(Sheets)) + 1
        Dim snbrSheets As String = Convert.ToString(nbrSheets)
        Debug.Print("Number of sheets: " + snbrSheets)
        Debug.Print(" ")

        Dim i As Integer
        For i = 0 To UBound(Sheets)
            Sheet = Sheets(i)
            Debug.Print("Name of sheet: " + (Sheet.Name))
            Debug.Print("Name of preview PNG's stream: " + (Sheet.PreviewPNGStreamName))
            Dim objBitMap As Object = Sheet.GetPreviewPNGBitmap(nError)
            If nError = 0 Then
                ' For each sheet, convert the picture to an
                ' image and save it as .png file
                Dim imgPreview As System.Drawing.Image = PictureDispConverter.Convert(objBitMap)
                imgPreview.Save("c:\temp\" + Sheet.Name + ".png", Drawing.Imaging.ImageFormat.Png)
                imgPreview.Dispose()

                ' Get the preview's PNG byte array
                PreviewPNGByteArray = Sheet.GetPreviewPNGBitmapBytes(nError)
                Dim nbrPreviewPNGBitmapBytes As Integer
                nbrPreviewPNGBitmapBytes = CLng(UBound(PreviewPNGByteArray))
                nbrPreviewPNGBitmapBytes = nbrPreviewPNGBitmapBytes + 1
                Dim snbrPreviewPNGBitmapBytes As String = Convert.ToString(nbrPreviewPNGBitmapBytes)
                Debug.Print("Number of bytes in preview's PNG byte array: " + snbrPreviewPNGBitmapBytes)
                Debug.Print("")
            Else
                Select Case nError
                    Case 2
                        Debug.Print("Error: No preview data stored with document.")
                    Case 4
                        Debug.Print("Error: Failed to make the bitmap.")
                End Select
            End If
        Next
    End Sub
End Module

```

```
        swDoc.CloseDoc()
    End Sub
End Module

'Class1
Public Class PictureDispConverter

    Inherits System.Windows.Forms.AxHost

    Public Sub New()
        MyBase.New("56174C86-1546-4778-8EE6-B6AC606875E7")
    End Sub

    Public Shared Function Convert(ByVal objIDispImage As Object) As System.Drawing.Image
        Dim objPicture As System.Drawing.Image
        objPicture = CType(System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage), System.Drawing.Image)
        Return objPicture
    End Function
End Class
```

Get Revision Tables Example (C#)

This example shows how to get a revision table in a drawing document, add a revision row, get revision row information, and update the revision row and column data.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API
//    Getting Started topic and ensure that
//    the required DLLs have been registered.
// 2. Copy and paste this module into a C#
//    console application in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll
//    reference to the project.
// 4. Substitute your_license_code with your SOLIDWORKS
//    Document Manager license key.
// 5. Ensure the specified document exists.
// 6. Compile and run this program in Debug mode.
//
// Postconditions:
// 1. Prints to the Immediate window:
//    * Path and filename
//    * Last saved date
//    * For each revision table:
//      * Name
//      * Sheet name
//      * Latest revision
//      * Revision information
//      * Revision table style
//      * Whether a placeholder row exists
// 2. In the revision table of sheet 1:
//    a. Adds a revision row.
//    b. Updates the revision column data.
//    c. Updates the revision row data.
//
// NOTE: Do not save changes to the document, as it is used elsewhere.
//-----

using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

namespace ConsoleApplication2
{
    class Module1
    {
        static void Main()
        {
            SwDMClassFactory dmClassFact = default(SwDMClassFactory);
            SwDMAApplication3 dmDocMgr = default(SwDMAApplication3);
            ISwDMDocument15 dmDoc = default(ISwDMDocument15);
            ISwDMTable6 dmTable = default(ISwDMTable6);
            ISwDMSheet4 dmSheet = default(ISwDMSheet4);
            SwDmDocumentOpenError dmError = default(SwDmDocumentOpenError);
            string nameDrawing = "";

            dmClassFact = new SwDMClassFactory();
            dmDocMgr = (SwDMAApplication3)dmClassFact.GetApplication("your_license_key"); //Do not distribute
            nameDrawing = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\advdrawings\\foodprocessor.slddrw";

            //Get the SOLIDWORKS drawing document
            dmDoc = (ISwDMDocument15)dmDocMgr.GetDocument(nameDrawing, SwDmDocumentType.swDmDocumentDrawing, false, out dmError);
            Debug.Print("Document's full name: " + dmDoc.FullName);
            Debug.Print("Date document last saved: " + dmDoc.LastSavedDate);

            //Get the names of the revision tables in the document
            string[] tableNames = null;
            tableNames = (string[])dmDoc.GetTableNames(SwDmTableType.swDmTableTypeRevision);
            int i = 0;
            if ((tableNames != null))
            {
                for (i = 0; i <= tableNames.Length - 1; i++)
                {
                    dmTable = (SwDMTable6)dmDoc.GetTable(tableNames[i]);
                    Debug.Print(" Revision table name: " + dmTable.Name);

                    dmSheet = (SwDMSheet4)dmTable.Sheet;
                    if (dmSheet != null)
                    {
                        Debug.Print(" On sheet: " + dmSheet.Name);
                    }
                }
            }
        }
    }
}
```

Get Revision Tables Example (VB.NET)

This example shows how to get a revision table in a drawing document, add a revision row, get revision row information, and update the revision row and column data.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
'    Getting Started topic and ensure that the
'    required DLLs are registered.
' 2. Copy and paste this module into a VB.NET
'    console application in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll
'    reference to the project.
' 4. Substitute your license key for your_license_key in the code.
' 5. Ensure the specified document exists.
' 6. Compile and run this program in Debug mode.
'
' Postconditions:
' 1. Adds a revision row.
' 2. Prints to the Immediate window:
'    * Path and filename
'    * Last saved date
'    * For each revision table:
'        * Name
'        * Sheet name
'        * Latest revision
'        * Revision information
'        * Revision table style
'        * Whether a placeholder row exists
' 3. Updates the latest revision column data.
' 4. Updates the latest revision row data.
'
' NOTE: Do not save changes to the document, as it is used elsewhere.
'-----

```

```

Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

```

```
Module Module1
```

```

    Dim dmClassFact As SwDMClassFactory
    Dim dmDocMgr As SwDMAApplication3
    Dim dmDoc As ISwDMDocument15
    Dim dmTable As ISwDMTable5
    Dim dmSheet As ISwDMSheet4
    Dim custPropView As ISwDMView
    Dim dmError As SwDmDocumentOpenError
    Dim arrviews As Array
    Dim nameDrawing As String

```

```
Sub Main()
```

```

    dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
    dmDocMgr = dmClassFact.GetApplication("your_license_key") 'Do not distribute
    nameDrawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\advdrawings\foodprocessor.sldrw"

```

```

    'Get the SOLIDWORKS drawing document
    dmDoc = dmDocMgr.GetDocument(nameDrawing, SwDmDocumentType.SwDmDocumentDrawing, False, dmError)
    Debug.Print("Document's full name: " & dmDoc.FullName)
    Debug.Print("Date document last saved: " & dmDoc.LastSavedDate)

```

```

    'Get the names of the revision tables in the document
    Dim tableNames As Object
    tableNames = dmDoc.GetTableNames(SwDmTableType.SwDmTableTypeRevision)
    Dim i As Integer

```

```

    If Not IsNothing(tableNames) Then
        For i = 0 To UBound(tableNames)
            dmTable = dmDoc.GetTable(tableNames(i))
            Debug.Print(" Revision table name: " & dmTable.Name)

```

```

            dmSheet = dmTable.Sheet
            If Not IsNothing(dmSheet) Then

```

```

                Debug.Print(" On sheet: " & dmSheet.Name)

```

```
            End If

```

```
        Next

```

```
    End If

```

```
End Sub
```

```
End Module
```

Get Table Cell Text Example (C#)

This example shows how to get all of the cell text from a BOM table using the SOLIDWORKS Document Manager API.

```
// -----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Help
//    Getting Started topic and ensure that the
//    required DLLs are registered.
// 2. Open SOLIDWORKS and copy the code below to a C# macro.
// 3. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
//    interop assembly is loaded in the project.
//    (Right-click on the project in Project Explorer, click
//    Add Reference, click the interop assembly in the .NET
//    tab, or browse for the DLL in install_dir\api\redist.)
// 4. In SOLIDWORKS, create a part document with a BOM table.
// 5. Save the part and close it.
// 6. Substitute your license key for your_license_key in the code.
// 7. Substitute the new file name for sFile in the code.
// 8. Uncomment the appropriate swDoc line.
//
// Postconditions: The Immediate Window displays the BOM
// table row and column count and all of the table's cell
// text in row order.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

partial class SolidWorksMacro
{
    SwDMClassFactory classfac;
    SwDMApplication tapp;
    SwDMDocument13 swDoc;
    SwDMTable3 swTable;
    SwDmDocumentOpenError e;
    string sFile;
    String[] vTables;
    String[] vTabArr;
    SwDMTableError err;
    int row;
    int col;
    int j;
    public void Main()
    {
        classfac = new SwDMClassFactory();
        tapp = classfac.GetApplication("your_license_key");

        sFile = "C:\\temp\\Part1.SLDPRT";

        //get the SW document file
        swDoc = (SwDMDocument13)tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentPart, false, out e);
        //swDoc = (SwDMDocument13)tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentAssembly, false, out e)
        //swDoc = (SwDMDocument13)tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentDrawing, false, out e)

        Debug.Print("Doc Version is " + swDoc.GetVersion());
    }
}
```

```

Debug.Print("Doc name is " + swDoc.FullName);
vTables = (String[])swDoc.GetTableNames(SwDmTableType.swDmTableTypeBOM);
if ((vTables != null))
{
    swTable = (SwDmTable3)swDoc.GetTable((String)vTables[0]);
    if ((swTable != null))
    {
        Debug.Print("Table retrieved is " + vTables[0]);
        vTabArr = (String[])swTable.GetTableCellText(out err, out row, out col);
        Debug.Print("Row count is " + row);
        Debug.Print("Column count is " + col);
        Debug.Print("Table cell text:");
        for (j = 0; j <= vTabArr.GetUpperBound(0); j++)
        {
            Debug.Print(" " + vTabArr[j]);
        }
    }
}
swDoc.CloseDoc();
}
public SldWorks swApp;
}

```

Get Table Cell Text Example (VB.NET)

This example shows how to get all of the cell text from a BOM table using the SOLIDWORKS Document Manager API.

```
' -----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Help
'    Getting Started topic and ensure that the
'    required DLLs are registered.
' 2. Open SOLIDWORKS and copy the code below to a VB.NET macro.
' 3. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
'    interop assembly is loaded in the project.
'    (Right-click on the project in Project Explorer, click
'    Add Reference, click the interop assembly in the .NET
'    tab, or browse for the DLL in install_dir\api\redist.)
' 4. In SOLIDWORKS, create a part document with a BOM table.
' 5. Save the part and close it.
' 6. Substitute your license key for your_license_key in the code.
' 7. Substitute the new file name for sFile in the code.
' 8. Uncomment the appropriate swDoc line.
'
' Postconditions: The Immediate Window displays the BOM
' table row and column count and all of the table's cell
' text in row order.
' -----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro
    Dim classfac As SwDMClassFactory
    Dim tapp As SwDMAApplication
    Dim swDoc As SwDMDocument13
    Dim swTable As SwDMTable3
    Dim e As SwDmDocumentOpenError
    Dim sFile As String
    Dim vTables As Object
    Dim vTabArr As Object
    Dim err As SwDmTableError
    Dim row As Integer, col As Integer
    Dim i As Integer, j As Integer
    Sub Main()
        classfac = CreateObject("SwDocumentMgr.SwDMClassFactory")
        tapp = classfac.GetApplication("your_license_key") 'license needed please do not distribute this
        sFile = "C:\temp\Part1.SLDPRPT"

        'get the SW document file
        swDoc = tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentPart, False, e)
        'Set swDoc = tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentAssembly, False, e)
        'Set swDoc = tapp.GetDocument(sFile, SwDmDocumentType.swDmDocumentDrawing, False, e)

        Debug.Print("Doc Version is " & swDoc.GetVersion)
        Debug.Print("Doc name is " & swDoc.FullName)
        vTables = swDoc.GetTableNames(SwDmTableType.swDmTableTypeBOM)
        If Not IsNothing(vTables) Then
            swTable = swDoc.GetTable(vTables(0))
            If Not swTable Is Nothing Then
```

```

        Debug.Print("Table retrieved is " & vTables(0))
        vTabArr = swTable.GetTableCellText(err, row, col)
        Debug.Print("Row count is " & row)
        Debug.Print("Column count is " & col)
        Debug.Print("Table cell text:")
        For j = 0 To UBound(vTabArr)
            Debug.Print("    " & vTabArr(j))
        Next
    End If
End If
swDoc.CloseDoc()
End Sub

Public swApp As SldWorks
End Class

```

Get Whether Component Is Envelope And Excluded From BOM (C#)

This example shows how to find out if a component is an envelope and whether the component is included in the bill of materials (BOM).

```
//-----
// Preconditions:
// 1. Open SOLIDWORKS and copy the code below to a C# macro.
// 2. Ensure that the specified file exists.
// 3. Specify your_license_key.
// 4. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//
// NOTE: ISwDMComponent5::ExcludeFromBom and ISwDMComponent5::IsEnvelope print
// to the Immediate Window for each component.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using SwDocumentMgr;
using System.Diagnostics;
namespace ExampleCS.cspj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            SwDocumentMgr.SwDMClassFactory swClassFact;
            SwDocumentMgr.SwDMAApplication swDocMgr;
            SwDocumentMgr.SwDMDocument7 swDoc;
            SwDocumentMgr.SwDMConfigurationMgr swCfgMgr;
            SwDMConfiguration7 swConfiguration7;

            string sDocFileName;
            SwDocumentMgr.SwDmDocumentType nDocType;
            SwDocumentMgr.SwDmDocumentOpenError nRetVal;
            string sLicenseKey;
            sLicenseKey = "your_license_key";
            sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\98food
processor.sldasm";
            nDocType = SwDocumentMgr.SwDmDocumentType.swDmDocumentAssembly;
            swClassFact = new SwDMClassFactory();
            swDocMgr = swClassFact.GetApplication(sLicenseKey);
            swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal) as SwDMDocument7;
            swCfgMgr = swDoc.ConfigurationManager;
            string[] SWConfigNames;
            SWConfigNames = (string[])swCfgMgr.GetConfigurationNames();
            for (int j = 0; j < SWConfigNames.Length; j++)
            {
                swConfiguration7 = swCfgMgr.GetConfigurationByName(SWConfigNames[j]) as SwDMConfiguration7;
                object[] swComponents;
                swComponents = (object[])swConfiguration7.GetComponents();
                Debug.Print("Configuration Name = " + SWConfigNames[j]);
                for (int I = 0; I < swComponents.Length; I++)
                {
                    SwDMComponent5 comp;
                    comp = swComponents[I] as SwDMComponent5;
                    Debug.Print("Component Name = " + comp.Name);
                    Debug.Print("        IsEnvelope = " + comp.IsEnvelope().ToString());
                }
            }
        }
    }
}
```

```
        Debug.Print("        ExcludeFromBOM = " + comp.ExcludeFromBOM.ToString());
    }
    Debug.Print("_____");
}
swDoc.CloseDoc();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

}
}
```

Get Whether Component Is Envelope And Excluded From BOM (VB.NET)

This example shows how to find out if a component is an envelope and whether the component is included in the bill of materials (BOM).

```
'-----
' Preconditions:
' 1. Open SOLIDWORKS and copy the code below to a VB.NET macro.
' 2. Ensure that the specified file exists.
' 3. Specify your_license_key.
' 4. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'
' NOTE: ISwDMComponent5::ExcludeFromBom and ISwDMComponent5::IsEnvelope print
' to the Immediate Window for each component.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports SwDocumentMgr
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Public Sub main()
        Dim swClassFact As SwDocumentMgr.SwDMClassFactory
        Dim swDocMgr As SwDocumentMgr.SwDMAApplication
        Dim swDoc As SwDocumentMgr.SwDMDocument7
        Dim swCfgMgr As SwDocumentMgr.SwDMConfigurationMgr
        Dim swConfiguration7 As SwDMConfiguration7
        Dim swComponents As Object
        Dim sDocFileName As String
        Dim nDocType As SwDocumentMgr.SwDmDocumentType
        Dim nRetVal As SwDocumentMgr.SwDmDocumentOpenError
        Dim sLicenseKey As String
        sLicenseKey = "your_license_key"
        sDocFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food
processor.sldasm"
        nDocType = SwDocumentMgr.SwDmDocumentType.swDmDocumentAssembly
        swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
        swDocMgr = swClassFact.GetApplication(sLicenseKey)
        swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)
        swCfgMgr = swDoc.ConfigurationManager
        Dim SWConfigNames() As String
        SWConfigNames = swCfgMgr.GetConfigurationNames
        For j As Integer = 0 To UBound(SWConfigNames)
            swConfiguration7 = swCfgMgr.GetConfigurationByName(SWConfigNames(j))
            swComponents = swConfiguration7.GetComponents
            Debug.Print("Configuration Name = " & SWConfigNames(j))
            For I As Integer = 0 To UBound(swComponents)
                Dim comp As SwDMComponent5
                comp = swComponents(I)
                Debug.Print("Component Name = " & comp.Name)
                Debug.Print("        IsEnvelope = " & comp.IsEnvelope)
                Debug.Print("        ExcludeFromBOM = " & comp.ExcludeFromBOM)
            Next
            Debug.Print("_____")
        Next
        swDoc.CloseDoc()
    End Sub
Public swApp As SldWorks
```

End Class

Get Whether Part Has Frozen Features Example (C#)

This example shows how to get whether a part document has frozen features, and, if so, whether those features need updating.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Help Getting Started
//    topic and register the required DLLs.
// 2. Copy and paste this module into a C#
//    console application in Microsoft Visual Studio.
// 3. Load the latest SolidWorks.Interop.swdocumentmgr.dll
//    interop assembly by:
//    a. Right-clicking the project name in the Project Explorer.
//    b. Clicking Add Reference.
//    c. Clicking the interop assembly in the .NET
//       tab or browsing for the DLL in install_dir/api/redist.
// 4. Substitute your license key for your_license_key in the code.
// 5. Make sure that the part document opened by this application
//    exists.
// 6. Open the Immediate window.
//
// Postconditions: Whether the part document contains frozen features
// is printed to the Immediate window, and, if so, whether
// those features need updating.
//
// NOTE: This sample application was developed using
// Microsoft Visual Studio 2008. If you use another version of
// Microsoft Visual Studio, you might need to add more references to get
// this application to compile.
//-----

using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

namespace HasFrozenFeatures
{
    class Module1
    {
        static void Main()
        {
            SwDMClassFactory dmClassFact = default(SwDMClassFactory);
            ISwDMAApplication3 dmDocMgr = default(ISwDMAApplication3);
            ISwMDMDocument16 dmDoc = default(ISwMDMDocument16);
            SwDmDocumentOpenError dmError = default(SwDmDocumentOpenError);
            string namePart = "";
            bool hasFrozenFeatures = false;
            bool frozenFeaturesNeedUpdating = false;

            dmClassFact = new SwDMClassFactory();
            //Do not distribute license key
            dmDocMgr = (ISwDMAApplication3)dmClassFact.GetApplication("your_license_key");
            namePart = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\FreezeBarNeedsRebuild2.sldprt";

            //Get the SOLIDWORKS part document
            dmDoc = (ISwMDMDocument16)dmDocMgr.GetDocument(namePart, SwDmDocumentType.swDmDocumentPart, false, out dmError);
            Debug.Print("Document's full name: " + dmDoc.FullName);
            Debug.Print("  Date document last saved: " + dmDoc.LastSavedDate);

            // Get whether the part document has frozen features and
            // whether those features need updating
            hasFrozenFeatures = (bool)dmDoc.HasFrozenFeatures(out frozenFeaturesNeedUpdating);
            Debug.Print("    Does the part document have frozen features? " + hasFrozenFeatures);
            if (hasFrozenFeatures)
                Debug.Print("      Do any of the frozen features need updating? " + frozenFeaturesNeedUpdating);
        }
    }
}
```

Get Whether Part Has Frozen Features Example (VB.NET)

This example shows how to get whether a part document has frozen features, and, if so, whether those features need updating.

```
'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Help Getting Started
'    topic and register the required DLLs.
' 2. Copy and paste this module into a VB.NET
'    console application in Microsoft Visual Studio.
' 3. Load the latest SolidWorks.Interop.swdocumentmgr.dll
'    interop assembly by:
'    a. Right-clicking the project name in the Project Explorer.
'    b. Clicking Add Reference.
'    c. Clicking the interop assembly in the .NET
'       tab or browsing for the DLL in install_dir/api/redist.
' 4. Substitute your license key for your_license_key in the code.
' 5. Make sure that the part document opened by this application
'    exists.
' 6. Open the Immediate window.
'
' Postconditions: Whether the part document contains frozen features
' is printed to the Immediate window, and, if so, whether
' those features need updating.
'
' NOTE: This sample application was developed using
' Microsoft Visual Studio 2008. If you use another version of
' Microsoft Visual Studio, you might need to add more references to get
' this application to compile.
'-----
```

```
Imports SolidWorks.Interop.swdocumentmgr
```

```
Imports System.Diagnostics
```

```
Module Module1
```

```
    Sub Main()
```

```
        Dim dmClassFact As SwDMClassFactory = Nothing
        Dim dmDocMgr As ISwDMAApplication3 = Nothing
        Dim dmDoc As ISwDMDocument16 = Nothing
        Dim dmError As SwDmDocumentOpenError = Nothing
        Dim namePart As String = ""
        Dim hasFrozenFeatures As Boolean = False
        Dim frozenFeaturesNeedUpdating As Boolean = False
```

```
        dmClassFact = New SwDMClassFactory()
```

```
        'Do not distribute license key
```

```
        dmDocMgr = DirectCast(dmClassFact.GetApplication("your_license_key"), ISwDMAApplication3)
```

```
        namePart = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\FreezeBarNeedsRebuild2.sldprt"
```

```
        'Get the SOLIDWORKS part document
```

```
        dmDoc = DirectCast(dmDocMgr.GetDocument(namePart, SwDmDocumentType.swDmDocumentPart, False, dmError), ISwDMDocument16)
```

```
        Debug.Print("Document's full name: " + dmDoc.FullName)
```

```
        Debug.Print(" Date document last saved: " + dmDoc.LastSavedDate)
```

```
        ' Get whether the part document has frozen features and
```

```
        ' whether those features need updating
```

```
        hasFrozenFeatures = CBool(dmDoc.HasFrozenFeatures(frozenFeaturesNeedUpdating))
```

```
        Debug.Print(" Does the part document have frozen features? " & hasFrozenFeatures)
```

```
        If hasFrozenFeatures Then
```

```
            Debug.Print(" Do any of the frozen features need updating? " & frozenFeaturesNeedUpdating)
```

```
        End If
```

```
    End Sub
```

```
End Module
```

Get XML Stream Example (C#)

This example shows how to get an embedded XML stream in a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API
//    Getting Started topic and ensure that the required
//    DLLs have been registered.
// 2. Copy and paste this module into a C# console
//    application in Microsoft Visual Studio.
// 3. Ensure that the specified document exists
//    (or point to another document).
// 4. Add the SolidWorks.Interop.swdocumentmgr.dll
//    reference to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click:
//        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
// 5. Substitute your_license_code with your
//    SOLIDWORKS Document Manager license key.
// Postconditions: Locate the XML stream in the current directory.
//-----

using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.swdocumentmgr;
class Program
{
    static void Main(string[] args)
    {
        const string sLicenseKey = "your_license_code";
        //Specify your license key
        const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\EDraw\\claw\\claw-mechanism.sldasm";

        SwDMClassFactory swClassFact = default(SwDMClassFactory);
        SwDMApplication swDocMgr = default(SwDMApplication);
        SwDMDocument swDoc = default(SwDMDocument);
        SwDmDocumentType nDocType = 0;
        string sPathName = null;
        SwDmDocumentOpenError nRetVal = 0;
        SwDmXmlDataError nRetVal2 = 0;

        // Determine type of SOLIDWORKS file based on file extension

        if (sDocFileName.EndsWith(".sldprt"))
        {
            nDocType = SwDmDocumentType.swDmDocumentPart;
        }
        else if (sDocFileName.EndsWith(".sldasm"))
        {
            nDocType = SwDmDocumentType.swDmDocumentAssembly;
        }
        else if (sDocFileName.EndsWith(".slddrw"))
        {
            nDocType = SwDmDocumentType.swDmDocumentDrawing;
        }
        else
        {
            // Not a SOLIDWORKS file
            nDocType = SwDmDocumentType.swDmDocumentUnknown;

            // So cannot open
            return;
        }

        // Strip off SOLIDWORKS file extension (sldxxx)
        // and add XML extension (xml)

        sPathName = sDocFileName.Substring(0, sDocFileName.Length - 6);
        sPathName = sPathName + ".xml";

        swClassFact = new SwDMClassFactory();
        swDocMgr = (SwDMApplication)swClassFact.GetApplication(sLicenseKey);
        swDoc = (SwDMDocument)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);

        swDoc.GetXmlStream(sPathName, ref nRetVal2);
    }
}
```

Get XML Stream Example (VB.NET)

This example shows how to get an embedded XML stream in a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

```
'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
'    Getting Started topic and ensure that the required
'    DLLs have been registered.
' 2. Copy and paste this module into a VB.NET console
'    application in Microsoft Visual Studio.
' 3. Ensure that the specified document exists
'    (or point to another document).
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll
'    reference to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click:
'        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 5. Substitute your_license_code with your
'    SOLIDWORKS Document Manager license key.
'-----
' Postconditions: Locate the XML stream in the current directory.
'-----
```

```
Imports SolidWorks.Interop.swdocumentmgr
Module Module1
```

```
    Sub main()

        Const sLicenseKey As String = "your_license_code" 'Specify your license key
        Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\EDraw\claw\claw-
mechanism.sldasm"

        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMAApplication
        Dim swDoc As SwDMDocument
        Dim nDocType As Integer
        Dim sPathName As String
        Dim nRetVal As Integer
        Dim nRetVal2 As Integer

        ' Determine type of SOLIDWORKS file based on file extension
        If InStr(LCase(sDocFileName), "sldprt") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentPart
        ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentAssembly
        ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentDrawing
        Else

            ' Probably not a SOLIDWORKS file
            nDocType = SwDmDocumentType.swDmDocumentUnknown

            ' so cannot open
            Exit Sub
        End If

        ' Strip off SOLIDWORKS file extension (sldxxx)
        ' and add XML extension (xml)

        sPathName = Left(sDocFileName, Len(sDocFileName) - 6)
        sPathName = sPathName + ".xml"

        swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
        swDocMgr = swClassFact.GetApplication(sLicenseKey)
        swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal)

        swDoc.GetXmlStream(sPathName, nRetVal)

    End Sub

End Module
```

GettingStarted-swdocmgrapi (Topic)

SOLIDWORKS Document Manager API Help

Getting Started

[Send Feedback](#)

This topic provides information to help you get started with using the SOLIDWORKS Document Manager API.

- [License Key](#)
- [Software Requirements](#)
- [Installation](#)
- [Application Basics](#)
- [Third-party Data in SOLIDWORKS Design Files](#)
- Read how you might use our [Active Content Protection Framework](#) to guard against malicious active content.

License Key

The SOLIDWORKS Document Manager API requires a license key that is only available via the SOLIDWORKS customer portal to SOLIDWORKS customers who are currently under subscription. Each user of SOLIDWORKS Document Manager API must have a license key.

Existing users must request new SOLIDWORKS Document Manager API license keys beginning with SOLIDWORKS Document Manager API 2015 FCS and renew their SOLIDWORKS Document Manager API license keys when installing any new major release of SOLIDWORKS or the SOLIDWORKS Document Manager API.

SOLIDWORKS Document Manager API checks the version of the file being accessed. If the file was created using the same or an earlier version of SOLIDWORKS than the version of SOLIDWORKS Document Manager API license key, then SOLIDWORKS Document Manager API can access the file. However, if the file was created using a later version of SOLIDWORKS than the version of the SOLIDWORKS Document Manager API license key, then SOLIDWORKS Document Manager API cannot access the file and a message is displayed informing the user that the current SOLIDWORKS Document Manager API license key is expired. For example:

Version in which SOLIDWORKS file created	Version of SOLIDWORKS Document Manager API license key	File Access Result
2014 or earlier	2014 or earlier	Success
2015 or later	2014 or earlier	Failure
2015 or earlier	2015 or later	Success

When requesting or renewing a SOLIDWORKS Document Manager API license key, you are presented with a dialog containing a list of check boxes showing the categories of functionality that a SOLIDWORKS Document Manager API license key controls. You can select more than one category.

- **Basic:** All interfaces, methods, and properties except for those listed below
- **Previews:** Preview and stream methods and properties on all ISwDMConfiguration, ISwDMDocument, and ISwDMSheet interfaces
- **DimXpert:** DimXpert methods and properties on all ISwDMConfiguration and all DimXpert-related interfaces
- **Geometry Streams:** Preview, stream, and imported body methods and properties on all ISwDMConfiguration interfaces
- **XML Streams:** XML stream methods on all ISwDMDocument interfaces
- **Tessellation:** Display List DLL (previously called the Display List Sample); although not a component of the SOLIDWORKS Document Manager API, its license key controls access to the Display List DLL

To request a SOLIDWORKS Document Manager API license key: <https://www.solidworks.com/support/subscription/key-request/>

A SOLIDWORKS Document Manager API license key is emailed to you upon approval of your request.

IMPORTANT: Do not share this license key with anyone outside your company or distribute it with any software that you ship.

You must specify the license key when instantiating a SOLIDWORKS Document Manager API connection. See [Application Basics](#) for details.

[Back to top](#)

Software Requirements

The SOLIDWORKS Document Manager API requires:

- [Visual C++ Redistributable for Visual Studio 2015](#), which installs the runtime components of Visual C++ libraries. These runtime components are required to successfully register the SOLIDWORKS Document Manager 2018 DLL, which was developed using Visual Studio 2015.
- Microsoft Core XML Services (MSXML) 6.0 or later; MSXML 6.0 is the first version of MSXML that supports 64-bit processors. MSXML is only needed if you want to use ISwDMComponent or ISwDMComponent2. Download MSXML from www.microsoft.com

NOTE: A full SOLIDWORKS 2018 installation includes MSXML and the Visual C++ Redistributable for Visual Studio 2015. Clients installing SOLIDWORKS Document Manager on systems without a full SOLIDWORKS 2018 installation must also install MSXML and the Visual C++ Redistributable or ensure that they have been previously installed.

[Back to top](#)

Installation

Read the Software Requirements section before proceeding. MSXML and the Visual C++ Redistributable for Visual Studio 2015 must be installed in order to successfully register the SOLIDWORKS Document Manager DLLs.

The SOLIDWORKS Document Manager API is a COM object contained in **SwDocumentMgr.dll**. The SOLIDWORKS Document Manager DimXpert API is contained in **dimxpert.dll**. The **zlib.dll** DLL was automatically installed in the following folder if you installed the SOLIDWORKS software:

<disk>:\Program Files\Common Files\SOLIDWORKS Shared

SwDocumentMgr.dll and **dimxpert.dll** were also automatically registered if you installed the SOLIDWORKS software.

If you did not install the SOLIDWORKS Design software on the system where you want to use these APIs, you can manually register them.

SwDocumentMgr.dll and **zlib.dll** must either reside in the same folder or be accessible via the Windows search path.

To manually register **SwDocumentMgr.dll**:

1. On the task bar, click the search icon.
2. Type "C" in the Type here to search field.
3. Right-click on Command Prompt, and select Run as administrator.
4. Click **Yes** in the dialog box.
5. In the Administrator Command Prompt, type:

```
regsvr32 "<disk>:\Program Files\Common Files\SOLIDWORKS Shared\SwDocumentMgr.dll"
```
6. Click **OK**.

NOTE: Install programs, such as InstallShield, can automatically register the DLL for you.

To manually unregister **SwDocumentMgr.dll**:

1. Follow steps 1-4 in the manual registration instructions above.
2. In the Administrator Command Prompt, type:

```
regsvr32 /u "<disk>:\Program Files\Common Files\SOLIDWORKS Shared\SwDocumentMgr.dll"
```
3. Click **OK**.

You can redistribute **swDocumentMgr.dll** and **zlib.dll**.

To manually register **dimxpert.dll**:

1. On the task bar, click the search icon.
2. Type "C" in the Type here to search field.
3. Right-click on Command Prompt, and select Run as administrator.
4. Click **Yes** in the dialog box.
5. In the Administrator Command Prompt, type (vX.X is the name of a version directory that contains **RegAsm.exe**):

```
C:\WINDOWS\Microsoft.NET\Framework\vX.X\regasm /codebase "<disk>:\Program Files\Common Files\SOLIDWORKS Shared\dimxpert.dll"
```
6. Click **OK**.

[Back to top](#)

Application Basics

Writing a SOLIDWORKS Document Manager application typically involves:

1. Instantiating a SOLIDWORKS Document Manager connection ([ISwDMClassFactory::GetApplication](#)).

NOTE: You must have a [license key](#) from SOLIDWORKS Corporation to instantiate the application object.

- ## 2. Opening a file ([ISwDMAApplication::GetDocument](#))

3. Querying the file for data such as custom properties, configurations, references, and where the file is used ([ISwDMConfiguration::GetBodiesCount](#), [ISwDMConfiguration::GetCustomPropertyNames](#), [ISwDMDocument::GetAllExternalReferences](#), [ISwDMDocument::GetHyperLinkAt](#)).

4. Extracting data such as Parasolid bodies and XML data from assemblies ([ISwDMConfiguration::GetBody](#) and [ISwDMDocument::GetXMLStream](#)).

5. Extracting features and annotations from DimXpert parts ([ISwDMConfiguration12:DimXpertPart](#)).

6. Making changes ([ISwDMDocument::AddCustomProperty](#), [ISwDMDocument::DeleteCustomProperty](#), [ISwDMDocument::ModifyHyperLinkAt](#), [ISwDMDocument::ReplaceReference](#), [ISwDMDocument::SetCustomProperty](#), etc.).

7. If any changes are made, saving the document ([ISwDMDocument::Save](#) or [ISwDMDocument::SaveAs](#)).

NOTE: If recreating an application using an example and the example uses a SOLIDWORKS sample file, do not save changes.

8. Closing the document ([ISwDMDocument::CloseDoc](#)).

NOTE: You must close a document before you can copy or move it using [ISwDMApplication::CopyDocument](#) or [ISwDMApplication::MoveDocument](#).

Be sure to examine all returned error codes.

NOTE: Documents created using SOLIDWORKS 99 (Version 1137) or earlier are not supported by this API. See [ISwDMDocument::GetVersion](#) for a detailed list of SOLIDWORKS versions supported by this API.

[Back to top](#)

Third-party Data in SOLIDWORKS Files

Starting with SOLIDWORKS 2015, third-party data stored in SOLIDWORKS Design files cannot be externally read or written by standard structured storage or compound file techniques.

To read or write third-party data, use the methods and events in the SOLIDWORKS API or SOLIDWORKS Document Manager API.

SOLIDWORKS API:

- [illegible]

SOLIDWORKS Document Manager API:

- [ISwDMDocument20::Delete3rdPartyStorage](#)
- [ISwDMDocument20::Delete3rdPartyStorageStore](#)
- [ISwDMDocument19::Get3rdPartyStorage](#)
- [ISwDMDocument19::Get3rdPartyStorageStore](#)
- [ISwDMDocument19::Release3rdPartyStorage](#)
- [ISwDMDocument19::Release3rdPartyStorageStore](#)

[Back to top](#)

Open and Close Document Example (C++)

This examples shows how to instantiate a SOLIDWORKS Document Manager session, open an assembly, get its latest supported SOLIDWORKS version number, list its external references, and close it.

```
#define WIN32_LEAN_AND_MEAN // Exclude rarely used stuff from Windows headers
#include <stdio.h>
#include <tchar.h>
#define _ATL_CSTRING_EXPLICIT_CONSTRUCTORS //Some CString constructors will be explicit
#include <windows.h>
#include <atlbase.h>
#include <iostream>
#include 'safearray.h'

using namespace std; //Use the standard C++ libraries for text output


#import 'C:\Program Files\Common Files\SOLIDWORKS Shared\swdocumentmgr.dll' raw_interfaces_only, raw_native_types, no_namespace, named_guids //Change as necessary

int _tmain(int argc, _TCHAR* argv[])
{
    //Initialize COM
    CoInitialize(NULL);


    CComPtr<ISwDMClassFactory> swClassFact;
    CComPtr<ISwDMApplication> swDocMgr;
    HRESULT hres = swClassFact.CoCreateInstance(__uuidof(SwDMClassFactory), NULL, CLSCTX_INPROC_SERVER );
    CComBSTR Key = _T("your_license_key"); //Specify your license key
    CComBSTR AsmDoc = _T("E:\\Assemblies\\Asm\\Assem1.sldasm"); //Replace with the name of your assembly (assuming it has one or more external references)
    swClassFact->GetApplication(Key, &swDocMgr);
    long ver = -1;

    swDocMgr->GetLatestSupportedFileVersion(&ver);
    cout <<"The latest supported file version is: " << ver << endl;


    CComPtr<ISwDMDocument> swDoc;
    SwDmDocumentOpenError res;
    swDocMgr->GetDocument(AsmDoc, swDmDocumentAssembly,VARIANT_TRUE, &res, &swDoc);


    CComPtr<ISwDMSearchOption> pSrcOpt;
    VARIANT Files;
    swDocMgr->GetSearchOptionObject(&pSrcOpt);
    swDoc->GetAllExternalReferences(pSrcOpt, &Files);
    SafeBSTRArray FilesArray(&Files);
    CComBSTR Ref1 = FilesArray[0]; //First file reference (no particular order)


    swDoc->CloseDoc();


    //Release references
    pSrcOpt = NULL;
    swDoc = NULL;
    swDocMgr = NULL;
    swClassFact = NULL;


    //Uninitialize COM
    CoUninitialize();


    return 0;
}
```

Release Notes

[Send Feedback](#)

This topic provides you with quick access to the enhancements in SOLIDWORKS Document Manager API 2025.

Service Pack 2

[New interface and method](#)

ISwDMDocument31 Interface

- [ISwDMDocument31::GetSheetNamesUntrimmed](#)

Service Pack 0

[New interface and method](#)

ISwDMDocument30 Interface

- [ISwDMDocument30::SavedInLDREditMode](#)

Replace Component Example (C#)

This example shows how to replace a component with a different component.

```

'-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
//    topic and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a VB.NET console application
//    in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click:
//        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
//    e. Click Add.
//    f. Click Close.
// 4. Ensure that the model to open exists.
// 5. Substitute your_license_key with your SOLIDWORKS Document
//    Manager license key.
// 6. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The first instance of the shaft washer component is
'    replaced with a lock washer in the document.
'
' NOTE: The specified file should be backed up before running this macro,
' as it is used elsewhere. The macro uses a replacement that does not
' properly fit in the model.
'-----

using System;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;
namespace ConsoleApplication2
{
    class Program
    {
        static void Main(string[] args)
        {
            SwDMClassFactory swCf;
            swCf = new SwDMClassFactory();
            SwDMApplication swDocMgr;
            swDocMgr = (SwDMApplication)swCf.GetApplication("your_license_key");
            SwDMDocument12 swDoc12;
            SwDmDocumentOpenError res;
            SwDmDocumentType dt;
            dt = SwDmDocumentType.swDmDocumentAssembly;
            string filename;
            Debug.Print("Opening an assembly...");

            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\98food
processor.sldasm";
            swDoc12 = swDocMgr.GetDocument(filename, dt, false, out res) as SwDMDocument12;

            if (swDoc12 == null | (res != SwDmDocumentOpenError.swDmDocumentOpenErrorNone))
            {
                Debug.Print("Error opening file...");
                return;
            }

            Debug.Print("Getting the active configuration...");
            SwDMConfiguration8 activeConfig;
            SwDMConfigurationMgr configMgr;
            configMgr = swDoc12.ConfigurationManager;
            configMgr.GetConfigurationNames();
            activeConfig = configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName()) as SwDMConfiguration8;
            if (activeConfig == null)
            {
                Debug.Print("Error getting the active configuration...");
                return;
            }

            Debug.Print("Getting the components of the active configuration...");
            object[] vComponents;
            vComponents = (object[])activeConfig.GetComponents();
            SwDMComponent6 swDmComponent;

            int i;
            for (i = 0; i < vComponents.Length; i++)
            {
                swDmComponent = (SwDMComponent6)vComponents[i];
                if (swDmComponent.Name == "shaft washer")
                {

```

```
        bool bResult = swDmComponent.Replace("C:\\Program Files\\SOLIDWORKS  
Corp\\SOLIDWORKS\\samples\\tutorial\\smartcomponents\\lockwasher.sldprt", "Default", false);  
        Debug.Print("Replacing shaft washer with lock washer...");  
        Debug.Print(bResult.ToString());  
        break;  
    }  
}  
  
swDoc12.Save();  
swDoc12.CloseDoc();  
}  
}
```

Replace Component (VB.NET)

This example shows how to replace a component with a different component.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
'    topic and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
'    in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click:
'        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
'    e. Click Add.
'    f. Click Close.
' 4. Ensure that the model to open exists.
' 5. Substitute your_license_key with your SOLIDWORKS Document
'    Manager license key.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The first instance of the shaft washer component is
'    replaced with a lock washer in the document.
'
' NOTE: The specified file should be backed up before running this macro,
' as it is used elsewhere. The macro uses a replacement that does not
' properly fit in the model.
'-----

```

Imports System.Diagnostics

Imports SolidWorks.Interop.swdocumentmgr

Module Module1

Sub Main()

```

    Dim swCf As SwDMClassFactory
    swCf = New SwDMClassFactory()
    Dim swDocMgr As SwDMAApplication
    swDocMgr = DirectCast(swCf.GetApplication("your_license_key"), SwDMAApplication)
    Dim swDoc12 As SwDMDocument12
    Dim res As SwDmDocumentOpenError
    Dim dt As SwDmDocumentType
    dt = SwDmDocumentType.swDmDocumentAssembly
    Dim filename As String

    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food processor.sldasm"
    swDoc12 = TryCast(swDocMgr.GetDocument(filename, dt, False, res), SwDMDocument12)

    If swDoc12 Is Nothing Or (res <> SwDmDocumentOpenError.swDmDocumentOpenErrorNone) Then
        Debug.Print("Error opening file...")
        Return
    End If

    Debug.Print("Getting the active configuration...")
    Dim activeConfig As SwDMConfiguration8
    Dim configMgr As SwDMConfigurationMgr
    configMgr = swDoc12.ConfigurationManager
    configMgr.GetConfigurationNames()
    activeConfig = TryCast(configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName()), SwDMConfiguration8)
    If activeConfig Is Nothing Then
        Debug.Print("Error getting the active configuration...")
        Return
    End If

    Debug.Print("Getting the components of the active configuration...")
    Dim vComponents As Object()
    vComponents = DirectCast(activeConfig.GetComponents(), Object())
    Dim swDmComponent As SwDMComponent6

    Dim i As Integer
    For i = 0 To vComponents.Length - 1
        swDmComponent = DirectCast(vComponents(i), SwDMComponent6)
        If swDmComponent.Name = "shaft washer" Then
            Dim bResult As Boolean = swDmComponent.Replace("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\smartcomponents\lockwasher.sldprt", "Default", False)
            Debug.Print("Replacing shaft washer with lock washer...")
            Debug.Print(bResult.ToString())
            Exit For
        End If
    Next

    swDoc12.Save()

```

```
SwDoc12.CloseDoc()
```

```
End Sub
```

```
End Module
```

SolidWorks.Interop.swdocumentmgr Assembly

 SOLIDWORKS Document Manager API Assembly

 Namespaces

Namespace	Description
SolidWorks.Interop.swdocumentmgr	SOLIDWORKS Document Manager API

Welcome

[Send Feedback](#)

The SOLIDWORKS Document Manager API:

- Allows you to write applications that emulate file management tools to perform such tasks as renaming, replacing, and copying SOLIDWORKS Design documents.
- Is an independent software component that provides read-write functionality for SOLIDWORKS Design documents. It includes interfaces for the document and configurations. These interfaces have read-only and read-write properties and methods.

This SOLIDWORKS Document Manager API is COM-compliant and supports automation-compliant data types only. It allows you to develop your application without reading or writing binary data directly to the header portion of SOLIDWORKS Design files. In fact, it is not necessary to have the SOLIDWORKS Design software installed on your system to use this API. It is, however, necessary to obtain a [license key](#) for the SOLIDWORKS Document Manager API.

NOTE: All of the sample code supplied is provided on an as-is basis and is only intended to demonstrate ways of using the SOLIDWORKS Document Manager API. SOLIDWORKS makes no representations or warranties regarding these samples. Any licensed user of the SOLIDWORKS Document Manager API is free to use any or all of these samples in connection with building applications related to SOLIDWORKS Document Manager API and is granted a royalty-free, non-exclusive license for these samples or parts thereof. Intellectual property rights of the samples remain with SOLIDWORKS. Any confidentiality provisions of the SOLIDWORKS Document Manager API license apply to the samples. Some sample code might contain obsolete APIs.

CAUTION: The SOLIDWORKS Document Manager API is a powerful programming tool that, if used incorrectly, could corrupt your data. Only experienced API programmers knowledgeable about SOLIDWORKS Design document references should use the SOLIDWORKS Document Manager API. It is recommended that you create backup copies of files accessed by your application before running the application.

Write Parasolid Partition Stream to File Example (C++)

This example shows how to write a Parasolid partition stream to a file using the SOLIDWORKS Document Manager API.

```
#import 'C:\Program Files\Common Files\SOLIDWORKS Shared\swdocumentmgr.dll' raw_interfaces_only, raw_native_types, no_namespace, named_guids //Change as necessary
```

```
wchar_t* filename = "87-part.sldprt"
wchar_t* outputDir = "c:\\caddocs\\";
```

```
ISwDMClassFactoryPtr swClassFact;
swClassFact.CreateInstance(
    __uuidof(SwDMClassFactory) );
if (!!(ISwDMClassFactory*)swClassFact)
    return 0;
```

```
ISwDMAApplicationPtr swDocMgr(
    swClassFact->GetApplication("your_license_key") ); //Specify your license key
if (!!(ISwDMAApplication*)swDocMgr)
    return 0;
```

```
SwDmDocumentOpenError error;
ISwDMDocumentPtr swDoc( swDocMgr->GetDocument(
    filename, swDmDocumentPart, TRUE, &error ) );
if (!!(ISwDMDocument*)swDoc)
    return 0;
```

```
ISwDMConfigurationMgrPtr swCfgMgr(
    swDoc->ConfigurationManager );
if (!!(ISwDMConfigurationMgr*)swCfgMgr)
    return 0;
```

```
_variant_t names( swCfgMgr->GetConfigurationNames() );
SAFEARRAY* nameArray = names.parray;
for (int i=0; i<nameArray->rgsabound[0].cElements; ++i)
{
    _bstr_t str( ((BSTR*)(nameArray->pvData))[i] );
```

```
ISwDMConfigurationPtr swCfg(
    swCfgMgr->GetConfigurationByName(str) );
ISwDMConfiguration2* pswCfg2;
if (swCfg)
    swCfg->QueryInterface( __uuidof(ISwDMConfiguration2), (void **)&pswCfg2 );
else
    return 0;
```

```
{
    CString dir( outputDir );
    dir += "\\";
    dir += str;
    dir += ".xmp_bin";
    pswCfg2->GetPartitionStream( (LPCTSTR)dir );
}
}
```

Write Parasolid Partition Stream to File Example (C#)

This example shows how to write a Parasolid partition stream to a file for a part.

NOTE: To get the name of the stream and a preview bitmap of the sheet active when the drawing was last saved, use ISwDMDocument10::GetPreviewBitmap and ISwDMDocument10::PreviewStreamName. To get the name of the streams and the preview bitmaps for all sheets in a drawing, use ISwDMSheet2::GetPreviewPNGBitmap and ISwDMSheet2::PreviewPNGStreamName.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
//    topic and verify that the required DLLs have been registered.
// 2. Create a C# macro in SOLIDWORKS.
// 3. Copy Main class into SolidWorksMacro.cs.
//    a. Specify your_license_key.
//    b. Verify that the specified part exists.
//    c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
//        1. Right-click the project in the Project Explorer.
//        2. Click Add Reference.
//        3. Click Browse.
//        4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
//    d. Add references to System.Drawing and stdole.
// 3. Create a class and copy Class1 into Class1.cs.
// 4. Verify that c:\temp exists.
// 5. Open the Immediate Window.
//
// Postconditions:
// 1. Creates c:\temp\multi-inter.bmp and c:\temp\Default.xmp_bin.
// 2. Examine the c:\temp and the Immediate Window.
//-----
//Main class
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using SolidWorks.Interop.swdocumentmgr;
using System.Drawing;
using stdole;
using System.Windows.Forms;
using System.Diagnostics;

namespace Macro1CSharpMacro.csproj
{
    public partial class SolidWorksMacro
    {
        public enum SwDmDocumentVersion_e
        {
            SwDmDocumentVersionSOLIDWORKS_2000 = 1500,
            SwDmDocumentVersionSOLIDWORKS_2001 = 1750,
            SwDmDocumentVersionSOLIDWORKS_2001Plus = 1950,
            SwDmDocumentVersionSOLIDWORKS_2003 = 2200,
            SwDmDocumentVersionSOLIDWORKS_2004 = 2500,
            SwDmDocumentVersionSOLIDWORKS_2005 = 2800,
            SwDmDocumentVersionSOLIDWORKS_2006 = 3100,
            SwDmDocumentVersionSOLIDWORKS_2007 = 3400,
            SwDmDocumentVersionSOLIDWORKS_2008 = 3800,
            SwDmDocumentVersionSOLIDWORKS_2009 = 4100,
            SwDmDocumentVersionSOLIDWORKS_2010 = 4400,
            SwDmDocumentVersionSOLIDWORKS_2011 = 4700,
            SwDmDocumentVersionSOLIDWORKS_2012 = 5000,
            SwDmDocumentVersionSOLIDWORKS_2013 = 6000,
            SwDmDocumentVersionSOLIDWORKS_2014 = 7000,
            SwDmDocumentVersionSOLIDWORKS_2015 = 8000,
            SwDmDocumentVersionSOLIDWORKS_2016 = 9000,
            SwDmDocumentVersionSOLIDWORKS_2017 = 10000,
            SwDmDocumentVersionSOLIDWORKS_2018 = 11000,
            SwDmDocumentVersionSOLIDWORKS_2019 = 12000,
            SwDmDocumentVersionSOLIDWORKS_2020 = 13000,
            SwDmDocumentVersionSOLIDWORKS_2021 = 14000,
            SwDmDocumentVersionSOLIDWORKS_2022 = 15000,
            SwDmDocumentVersionSOLIDWORKS_2023 = 16000,
            SwDmDocumentVersionSOLIDWORKS_2024 = 17000,
            SwDmDocumentVersionSOLIDWORKS_2025 = 18000,
            SwDmDocumentVersionSOLIDWORKS_2026 = 19000
        }

    }

    public void Main()
    {
        const string sLicenseKey = "your_license_key";
        const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2024\\samples\\tutorial\\multibody\\multi_inter.sldprt";
        const string sExtractDir = "c:\\temp\\";

        SwDMClassFactory swClassFact = default(SwDMClassFactory);
        SwDMAApplication swDocMgr = default(SwDMAApplication);
        SwDMDocument10 swDoc = default(SwDMDocument10);
        SwDMConfigurationMgr swCfgMgr = default(SwDMConfigurationMgr);
        string[] vCfgNameArr = null;
        SwDMConfiguration2 swCfg = default(SwDMConfiguration2);
        SwDMDocumentType nDocType = 0;
        SwDmPreviewError nError = 0;
        SwDmBodyError nBodyError = 0;
        SwDmDocumentOpenError nRetVal = 0;
        int i = 0;
        string sFileName = "multi-inter";

        // Determine type of SOLIDWORKS file based on file extension
    }
}
```

```

if (sDocFileName.EndsWith("sldprt"))
{
    nDocType = SwDmDocumentType.swDmDocumentPart;
}
else if (sDocFileName.EndsWith("sldasm"))
{
    nDocType = SwDmDocumentType.swDmDocumentAssembly;
}
else if (sDocFileName.EndsWith("slddrw"))
{
    nDocType = SwDmDocumentType.swDmDocumentDrawing;
}
else
{
    // Not a SOLIDWORKS file, so cannot open
    nDocType = SwDmDocumentType.swDmDocumentUnknown;
    return;
}

// Because drawing documents do not have configurations,
// only continue running the macro if the document
// is a part or assembly document
if ((nDocType != SwDmDocumentType.swDmDocumentDrawing))
{
    swClassFact = new SwDMClassFactory();
    swDocMgr = (SwDMApplication)swClassFact.GetApplication(sLicenseKey);
    swDoc = (SwDMDocument10)swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal);

    swCfgMgr = swDoc.ConfigurationManager;

    Debug.Print("File = " + swDoc.FullName);
    Debug.Print(" Active Configuration Name = " + swCfgMgr.GetActiveConfigurationName());
    Debug.Print("");
    Debug.Print(" Version = " + swDoc.GetVersion());
    Debug.Print(" Author = " + swDoc.Author);
    Debug.Print(" Comments = " + swDoc.Comments);
    Debug.Print(" Creation Date = " + swDoc.CreationDate);
    Debug.Print(" Keywords = " + swDoc.Keywords);
    Debug.Print(" Last Saved By = " + swDoc.LastSavedBy);
    Debug.Print(" Last Saved Date = " + swDoc.LastSavedDate);
    Debug.Print(" Subject = " + swDoc.Subject);
    Debug.Print(" Title = " + swDoc.Title);

    vCfgNameArr = (string[])swCfgMgr.GetConfigurationNames();
    foreach (string vCfgName in vCfgNameArr)
    {
        swCfg = (SwDMConfiguration2)swCfgMgr.GetConfigurationByName(vCfgName);

        // SwDMConfiguration::GetPreviewBitmap throws an unmanaged COM exception
        // for out-of-process C# console applications
        // Use the following code in SOLIDWORKS C# macros and add-ins
        object objBitMap = swCfg.GetPreviewBitmap(out nError);
        System.Drawing.Image imgPreview = PictureDispConverter.Convert(objBitMap);
        imgPreview.Save("c:\\temp\\" + sFileName + ".bmp", System.Drawing.Imaging.ImageFormat.Bmp);
        imgPreview.Dispose();

        Debug.Print(" " + vCfgName);
        Debug.Print(" Description = " + swCfg.Description);
        Debug.Print(" Parent ConfigurationName = " + swCfg.GetParentConfigurationName());
        Debug.Print(" Preview stream = " + swCfg.PreviewStreamName);

        switch ((SwDmDocumentVersion_e)swDoc.GetVersion())
        {
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2000:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2001:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2001Plus:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2003:
                Debug.Print(" Body stream = " + swCfg.StreamName);
                Debug.Print(" Body count = " + (swCfg.GetBodiesCount() - 1));

                for (i = 0; i <= swCfg.GetBodiesCount() - 1; i++)
                {
                    nBodyError = swCfg.GetBody(i, sExtractDir + vCfgName + "_" + i.ToString() + ".x_b");
                }

                break;
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2004:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2005:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2006:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2007:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2008:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2009:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2010:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2011:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2012:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2013:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2014:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2015:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2016:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2017:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2018:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2019:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2020:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2021:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2022:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2023:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2024:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2025:
            case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2026:
                Debug.Print(" Partition stream = " + swCfg.StreamName);
                // SwDMConfiguration2::GetPartitionStream takes file name as an input and writes
                // to it. The extensions .xmp_bin (for NTFS) and .p_b (for FAT) are
                // the valid extensions of a partition file. PK_PARTITION_receive_u takes
                // this file name as input to get partitions and bodies in it.
                // Refer to Parasolid documentation for details.
                nBodyError = swCfg.GetPartitionStream(sExtractDir + vCfgName + ".xmp_bin");
                break;
        }
    }
}

```

```

        default:
            break;
    }
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

//Class1
public class PictureDispConverter : System.Windows.Forms.AxHost
{
    public PictureDispConverter()
        : base("56174C86-1546-4778-8EE6-B6AC606875E7")
    {
    }

    public static System.Drawing.Image Convert(object objIDispImage)
    {
        System.Drawing.Image objPicture = default(System.Drawing.Image);
        objPicture = (System.Drawing.Image)System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage);
        return objPicture;
    }
}

```

Write Parasolid Partition Stream to File Example_VBNET (Topic)

SOLIDWORKS Document Manager API Help

Write Parasolid Partition Stream to File Example (VB.NET)

This example shows how to write a Parasolid partition stream to a file for a part.

NOTE: To get the name of the stream and a preview bitmap of the sheet active when the drawing was last saved, use ISwDMDocument10::GetPreviewBitmap and ISwDMDocument10::PreviewStreamName. To get the name of the streams and the preview bitmaps for all sheets in a drawing, use ISwDMSheet2::GetPreviewPNGBitmap and ISwDMSheet2::PreviewPNGStreamName.

```
-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
'   topic and verify that the required DLLs are registered.
' 2. Create a VB.NET console application in Microsoft Visual Studio.
' 3. Copy and paste Module1 into Module1.vb.
'   a. Specify your_license_key.
'   b. Verify that the specified part to open exists.
'   c. Add a reference to SolidWorks.Interop.swdocumentmgr.dll:
'       1. Right-click the solution in the Solution Explorer.
'       2. Click Add Reference.
'       3. Click Browse.
'       4. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'   d. Add references to System.Windows.Forms, System.Drawing, and stdole.
' 4. Create a class and copy Class1 into Class1.vb.
' 5. Verify that c:\temp exists.
' 6. Open the Immediate Window.
' Postconditions:
' 1. Creates c:\temp\multi-inter.bmp and c:\temp\Default.xmp_bin.
' 2. Examine the c:\temp and the Immediate Window.
-----
'Module1
Imports System
Imports System.Diagnostics
Imports SolidWorks.Interop.swdocumentmgr
Imports System.Drawing
Imports stdole
Imports System.Windows.Forms

Module Module1

    Public Enum SwDmDocumentVersion_e
        SwDmDocumentVersionSOLIDWORKS_2000 = 1500
        SwDmDocumentVersionSOLIDWORKS_2001 = 1750
        SwDmDocumentVersionSOLIDWORKS_2001Plus = 1950
        SwDmDocumentVersionSOLIDWORKS_2003 = 2200
        SwDmDocumentVersionSOLIDWORKS_2004 = 2500
        SwDmDocumentVersionSOLIDWORKS_2005 = 2800
        SwDmDocumentVersionSOLIDWORKS_2006 = 3100
        SwDmDocumentVersionSOLIDWORKS_2007 = 3400
        SwDmDocumentVersionSOLIDWORKS_2008 = 3800
        SwDmDocumentVersionSOLIDWORKS_2009 = 4100
        SwDmDocumentVersionSOLIDWORKS_2010 = 4400
        SwDmDocumentVersionSOLIDWORKS_2011 = 4700
        SwDmDocumentVersionSOLIDWORKS_2012 = 5000
        SwDmDocumentVersionSOLIDWORKS_2013 = 6000
        SwDmDocumentVersionSOLIDWORKS_2014 = 7000
        SwDmDocumentVersionSOLIDWORKS_2015 = 8000
        SwDmDocumentVersionSOLIDWORKS_2016 = 9000
        SwDmDocumentVersionSOLIDWORKS_2017 = 10000
        SwDmDocumentVersionSOLIDWORKS_2018 = 11000
        SwDmDocumentVersionSOLIDWORKS_2019 = 12000
        SwDmDocumentVersionSOLIDWORKS_2020 = 13000
        SwDmDocumentVersionSOLIDWORKS_2021 = 14000
        SwDmDocumentVersionSOLIDWORKS_2022 = 15000
        SwDmDocumentVersionSOLIDWORKS_2023 = 16000
        SwDmDocumentVersionSOLIDWORKS_2024 = 17000
        SwDmDocumentVersionSOLIDWORKS_2025 = 18000
        SwDmDocumentVersionSOLIDWORKS_2026 = 19000
    End Enum

    Sub main()

        Const sLicenseKey As String = "your_license_key"
        Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\multibody\multi_inter.sldprt"
        Const sExtractDir As String = "c:\temp\"

        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMApplication
        Dim swDoc As SwDMDocument
        Dim swDoc10 As SwDMDocument10
        Dim swCfgMgr As SwDMConfigurationMgr
        Dim vCfgNameArr As Object
        Dim vCfgName As String
        Dim swCfg As SwDMConfiguration2
        Dim nDocType As Integer
        Dim nRetVal As Integer
        Dim i As Integer
        Dim sFileName As String = "multi_inter"

        ' Determine type of SOLIDWORKS file based on file extension
        If InStr(LCase(sDocFileName), "sldprt") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentPart
        ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentAssembly
        ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
            nDocType = SwDmDocumentType.swDmDocumentDrawing
        Else
            ' Probably not a SOLIDWORKS file, so cannot open
            nDocType = SwDmDocumentType.swDmDocumentUnknown
        End If
    End Sub
End Module
```

```

Exit Sub
End If

' Because drawing documents do not have configurations,
' only continue running the macro if the document
' is a part or assembly document
If (nDocType <> SwDmDocumentType.SwDmDocumentDrawing) Then
    swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
    swDocMgr = swClassFact.GetApplication(sLicenseKey)
    swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)

    swCfgMgr = swDoc.ConfigurationManager

    Debug.Print("File = " & swDoc.FullName)
    Debug.Print(" Active Configuration Name = " & swCfgMgr.GetActiveConfigurationName)
    Debug.Print("")
    Debug.Print(" Version = " & swDoc.GetVersion)
    Debug.Print(" Author = " & swDoc.Author)
    Debug.Print(" Comments = " & swDoc.Comments)
    Debug.Print(" Creation Date = " & swDoc.CreationDate)
    Debug.Print(" Keywords = " & swDoc.Keywords)
    Debug.Print(" Last Saved By = " & swDoc.LastSavedBy)
    Debug.Print(" Last Saved Date = " & swDoc.LastSavedDate)
    Debug.Print(" Subject = " & swDoc.Subject)
    Debug.Print(" Title = " & swDoc.Title)

    vCfgNameArr = swCfgMgr.GetConfigurationNames

    For Each vCfgName In vCfgNameArr
        swCfg = swCfgMgr.GetConfigurationByName(vCfgName)
        swDoc10 = swDoc
        Dim objBitMap As Object = swDoc10.GetPreviewBitmap(nRetVal)
        Dim imgPreview As Drawing.Image = PictureDispConverter.Convert(objBitMap)
        imgPreview.Save(sExtractDir + sFilename + ".bmp", Drawing.Imaging.ImageFormat.Bmp)
        imgPreview.Dispose()

        Debug.Print(" " & vCfgName)
        Debug.Print(" Description = " & swCfg.Description)
        Debug.Print(" Parent Configuration Name = " & swCfg.GetParentConfigurationName)
        Debug.Print(" Preview stream = " & swCfg.PreviewStreamName)

        Select Case swDoc.GetVersion

            Case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2000, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2001, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2001Plus, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2003
                Debug.Print(" Body stream = " & swCfg.StreamName)
                Debug.Print(" Body count = " & swCfg.GetBodiesCount - 1)

                For i = 0 To swCfg.GetBodiesCount - 1
                    nRetVal = swCfg.GetBody(i, sExtractDir & vCfgName & "_" & i & ".x_b")
                Next i

            Case SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2004, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2005, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2006, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2007, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2008, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2009, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2010, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2011, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2012, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2013, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2014, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2015, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2016, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2017, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2018, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2019, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2020, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2021, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2022, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2023, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2024, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2025, SwDmDocumentVersion_e.SwDmDocumentVersionSOLIDWORKS_2026
                Debug.Print(" Partition stream = " & swCfg.StreamName)

                ' SwDMConfiguration2::GetPartitionStream takes file name as an input and writes
                ' to it. The extensions .xmp_bin (for NTFS) and .p_b (for FAT) are the valid
                ' extensions of a partition file. PK_PARTITION_receive_u takes this file name as
                ' input to get partitions and bodies in it.
                ' Refer to Parasolid documentation for details.

                nRetVal = swCfg.GetPartitionStream(sExtractDir & vCfgName & ".xmp_bin")

            Case Else

        End Select
    Next
End If
End Sub
End Module

'Class1
Public Class PictureDispConverter
    Inherits System.Windows.Forms.AxHost

    Public Sub New()
        MyBase.New("56174C86-1546-4778-8EE6-B6AC606875E7")
    End Sub

    Public Shared Function Convert(ByVal objIDispImage As Object) As Drawing.Image
        Dim objPicture As Drawing.Image
        objPicture = CType(System.Windows.Forms.AxHost.GetPictureFromIPicture(objIDispImage), Drawing.Image)
        Return objPicture
    End Function
End Class

```